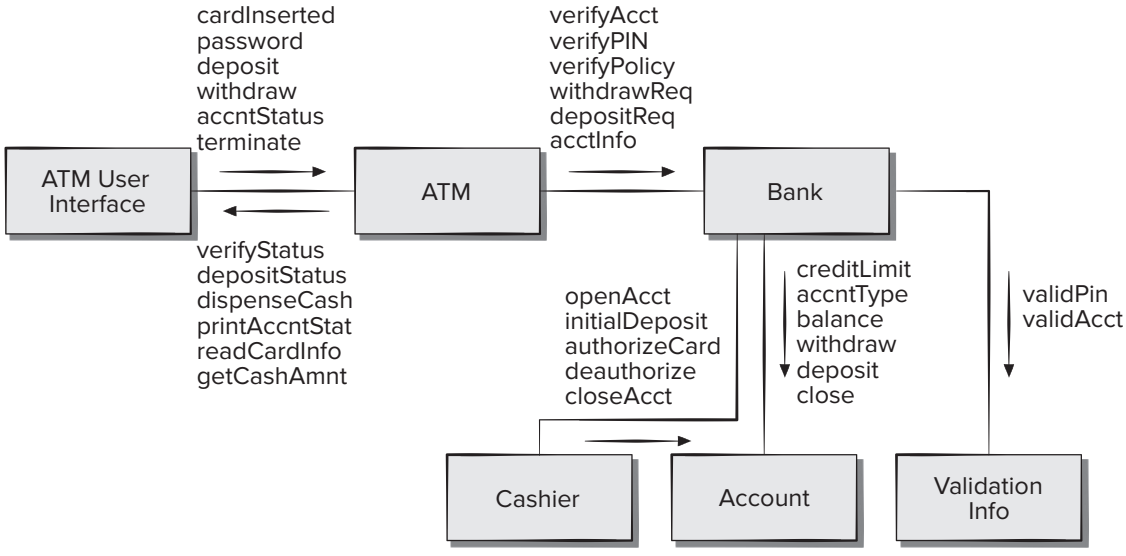
FIGURE 20.3 Class collaboration diagram for banking application



Source: Kirani, Shekhar and Tsai, W. T., "Specification and Verification of Object-Oriented Programs," Technical Report TR 94-64, University of Minnesota, December 4, 1994, 72.

3. For each operation in the server object (that has been invoked by messages sent from the client object), determine the messages that it transmits.
4. For each of the messages, determine the next level of operations that are invoked and incorporate these into the test sequence.

To illustrate [Kir94], consider a sequence of operations for the **Bank** class relative to an **ATM** class (Figure 20.3):

```
verifyAcct•verifyPIN•[[verifyPolicy•withdrawReq]|depositReq|acctInfoREQ]^n
```

A random test case for the **Bank** class might be

Test case r_3 = `verifyAcct•verifyPIN•depositReq`

To consider the collaborators involved in this test, the messages associated with each of the operations noted in test case r_3 are considered. **Bank** must collaborate with **ValidationInfo** to execute the *verifyAcct()* and *verifyPIN()*. **Bank** must collaborate with **Account** to execute *depositReq()*. Hence, a new test case that exercises these collaborations is

Test case r_4 = `verifyAcct [Bank:validAcctValidationInfo]•verifyPIN`
`[Bank: validPinValidationInfo]•depositReq [Bank: depositaccount]`

20.4.2 Scenario-Based Test-Case Design

Fault-based testing misses two main types of errors: (1) incorrect specifications and (2) interactions among subsystems. When errors associated with an incorrect specification occur, the product doesn't do what the customer wants. It might do the wrong

thing or omit important functionality. But in either circumstance, quality (conformance to requirements) suffers. Errors associated with subsystem interaction occur when the behavior of one subsystem creates circumstances (e.g., events, data flow) that cause another subsystem to fail.

Scenario-based testing will uncover errors that occur when any actor interacts with the software. Scenario-based testing concentrates on what the user does, not what the product does. This means capturing the tasks (via use cases) that the user has to perform and then applying them and their variants as tests. This is very similar to thread testing.

Scenario testing uncovers interaction errors. But to accomplish this, test cases must be more complex and more realistic than fault-based tests. Scenario-based testing tends to exercise multiple subsystems in a single test (users do not limit themselves to the use of one subsystem at a time).

Test-case design becomes more complicated as integration of the object-oriented system begins. It is at this stage that testing of collaborations between classes must begin. To illustrate "interclass test-case generation" [Kir94], we expand the banking example introduced in Section 19.6 to include the classes and collaborations noted in Figure 20.3. The direction of the arrows in the figure indicates the direction of messages, and the labeling indicates the operations that are invoked as a consequence of the collaborations implied by the messages.

Like the testing of individual classes, class collaboration testing can be accomplished by applying random and partitioning methods, as well as scenario-based testing and behavioral testing.

20.5 VALIDATION TESTING

Like all testing steps, validation tries to uncover errors, but the focus is at the requirements level—on things that will be immediately apparent to the end user. Validation testing begins at the culmination of integration testing, when individual components have been exercised, the software is completely assembled as a package, and interfacing errors have been uncovered and corrected. At the validation or system level, the distinction between different software categories disappears. Testing focuses on user-visible actions and user-recognizable output from the system.

Validation can be defined in many ways, but a simple (albeit harsh) definition is that validation succeeds when software functions in a manner that can be reasonably expected by the customer. At this point, a battle-hardened software developer might protest: "Who or what is the arbiter of reasonable expectations?" If a *software requirements specification* has been developed, it describes each user story, all user-visible attributes, and the customer's acceptance criteria for each. The customer's acceptance criteria form the basis for a validation-testing approach.

Software validation is achieved through a series of tests that demonstrate conformity with requirements. A test plan outlines the classes of tests to be conducted, and a test procedure defines specific test cases that are designed to ensure that all functional requirements are satisfied, all behavioral characteristics are achieved, all content is accurate and properly presented, all performance requirements are attained, documentation is correct, and usability and other requirements are met (e.g., transportability,

compatibility, error recovery, maintainability). If a deviation from specification is uncovered, a *deficiency list* is created. A method for resolving deficiencies (acceptable to stakeholders) must be established. Specialized testing methods for these nonfunctional requirements are discussed in Chapter 21.

An important element of the validation process is a *configuration review.* The intent of the review is to ensure that all elements of the software configuration have been properly developed, are cataloged, and have the necessary detail to bolster the support activities. The configuration review, sometimes called an audit, is discussed in more detail in Chapter 22.

SAFEHOME



Preparing for Validation

The scene: Doug Miller's office, as component-level design continues and construction of certain components continues.

The players: Doug Miller, software engineering manager; Vinod, Jamie, Ed, and Shakira, members of the *SafeHome* software engineering team.

The conversation:

Doug: The first increment will be ready for validation in what . . . about three weeks?

Vinod: That's about right. Integration is going well. We're smoke testing daily, finding some bugs, but nothing we can't handle. So far, so good.

Doug: Talk to me about validation.

Shakira: Well, we'll use all of the use cases as the basis for our test design. I haven't started yet, but I'll be developing tests for all of the use cases that I've been responsible for.

Ed: Same here.

Jamie: Me too, but we've got to get our act together for acceptance testing and also for alpha and beta testing, no?

Doug: Yes. In fact I've been thinking; we could bring in an outside contractor to help us with validation. I have the money in the budget . . . and it'd give us a new point of view.

Vinod: I think we've got it under control.

Doug: I'm sure you do, but an ITG gives us an independent look at the software.

Jamie: We're tight on time here, Doug. I for one don't have the time to babysit anybody you bring in to do the job.

Doug: I know, I know. But if an ITG works from requirements and use cases, not too much babysitting will be required.

Vinod: I still think we've got it under control.

Doug: I hear you, Vinod, but I am going to overrule on this one. Let's plan to meet with the ITG rep later this week. Get 'em started and see what they come up with.

Vinod: Okay, maybe it'll lighten the load a bit.

At the validation or system level, the details of class connections disappear. The validation of OO software focuses on user-visible actions and user-recognizable outputs from the system. To assist in the derivation of validation tests, the tester should draw upon use cases (Chapters 7 and 8) that are part of the requirements model. The use case provides a scenario that has a high likelihood of uncovered errors in user-interaction requirements. Conventional black-box testing methods (Chapter 19) can be used to drive validation tests. In addition, you may choose to derive test cases from the object-behavior model created as part of object-oriented analysis (OOA).

20.6 TESTING PATTERNS

The use of patterns as a mechanism for describing solutions to specific design problems was discussed in Chapter 15. But patterns can also be used to propose solutions to other software engineering situations—in this case, software testing. *Testing patterns* describe common testing problems and solutions that can assist you in dealing with them.

Much of software testing, even during the past decade, has been an ad hoc activity. If testing patterns can help a software team to communicate about testing more effectively, to understand the motivating forces that lead to a specific approach to testing, and to approach the design of tests as an evolutionary activity in which each iteration results in a more complete suite of test cases, then patterns have accomplished much.

Testing patterns are described in much the same way as design patterns (Chapter 15). Dozens of testing patterns have been proposed in the literature (e.g., [Mar02]). The following three testing patterns (presented in abstract form only) provide representative examples:

Pattern name: **PairTesting**

Abstract: A process-oriented pattern, **pair testing** describes a technique that is analogous to pair programming [Chapter 3] in which two testers work together to design and execute a series of tests that can be applied to unit, integration or validation testing activities.

Pattern name: **SeparateTestInterface**

Abstract: There is a need to test every class in an object-oriented system, including "internal classes" (i.e., classes that do not expose any interface outside of the component that used them). The **SeparateTestInterface** pattern describes how to create "a test interface that can be used to describe specific tests on classes that are visible only internally to a component" [Lan01].

Pattern name: **ScenarioTesting**

Abstract: Once unit and integration tests have been conducted, there is a need to determine whether the software will perform in a manner that satisfies users. The **ScenarioTesting** pattern describes a technique for exercising the software from the user's point of view. A failure at this level indicates that the software has failed to meet a user visible requirement [Kan01].

A comprehensive discussion of testing patterns is beyond the scope of this book. If you have further interest, see [Bin99], [Mar02], [Tho04], [Mac10], and [Gon17] for additional information on this important topic.

20.7 SUMMARY

Integration testing builds the software architecture while at the same time conducting tests to uncover errors associated with interfacing between software components. The objective is to take unit-tested components and build a program structure that has been dictated by design.

Experienced software developers often say, "Testing never ends; it just gets transferred from you [the software engineer] to your customer. Every time your customer uses the program, a test is being conducted." By applying test-case design, you can

achieve more complete testing and thereby uncover and correct the highest number of errors before the "customer's tests" begin.

Hetzel [Het84] describes white-box testing as "testing in the small." His implication is that the white-box tests that have been considered in this chapter are typically applied to small program components (e.g., modules or small groups of modules). Black-box testing, on the other hand, broadens your focus and might be called "testing in the large."

Black-box integration testing is based on the requirements specified in the user stories or some other analysis modeling representation. Test-case authors do not need to wait for the component implementation code to be written, as long as they understand the required functionality of the components undergoing testing. Validation testing is often accomplished with black-box test cases that produce end-user visible input actions and observable output behaviors.

White-box testing requires a close examination of procedural implementation details and data structure implementation details for the components undergoing testing. White-box tests can be designed only after component-level design (or source code) exists. Logical paths through the software and collaborations between components are the focus of white-box integration testing.

Integration testing of OO software can be accomplished using a thread-based or use-based strategy. Thread-based testing integrates the set of classes that collaborate to respond to one input or event. Use-based testing constructs the system in layers, beginning with those classes that do not make use of server classes. Integration test-case design methods can also make use of random and partition tests. In addition, scenario-based testing and tests derived from behavioral models can be used to test a class and its collaborators. A test sequence tracks the flow of operations across class collaborations.

OO system validation testing is black-box oriented and can be accomplished by applying the same black-box methods discussed for conventional software. However, scenario-based testing dominates the validation of OO systems, making the use case a primary driver for validation testing.

Regression testing is the process of reexecuting a selected test case following any change made to a software system. Regression tests should be executed whenever new components or changes are added to a software increment. Regression testing helps to ensure that changes do not introduce unintended behavior or additional errors.

Testing patterns describe common testing problems and solutions that can assist you in dealing with them. If testing patterns can help a software team to communicate about testing more effectively, to understand the motivating forces that lead to a specific approach to testing, and to approach the design of test cases as an evolutionary activity in which each iteration results in a more complete suite of test cases, then patterns have accomplished much.

PROBLEMS AND POINTS TO PONDER

20.1. How can project scheduling affect integration testing?

20.2. Who should perform validation testing—the software developer or the software user? Justify your answer.

20.3. Will exhaustive testing (even if it is possible for very small programs) guarantee that a program is 100 percent correct?

20.4. Why should "testing" begin with requirements analysis and design?

20.5. Should nonfunctional requirements (e.g., security or performance) be tested as part of integration testing?

20.6. Why do we have to retest subclasses that are instantiated from an existing class, if the existing class has already been thoroughly tested?

20.7. What is the difference between thread-based and use-based strategies for integration testing?

20.8. Develop a complete test strategy for the *SafeHome* system discussed earlier in this book. Document it in a *test specification.*

20.9. Pick one of the *SafeHome* system user stories to use as the basis of scenarios-based testing, and construct a set of integration test cases needed to do integration testing for that user story.

20.10. For the test cases you wrote in Problem 20.9, identify a subset of test cases you will use for regression testing software components that are added to the program.

Design element: Quick Look icon magnifying glass: © Roger Pressman